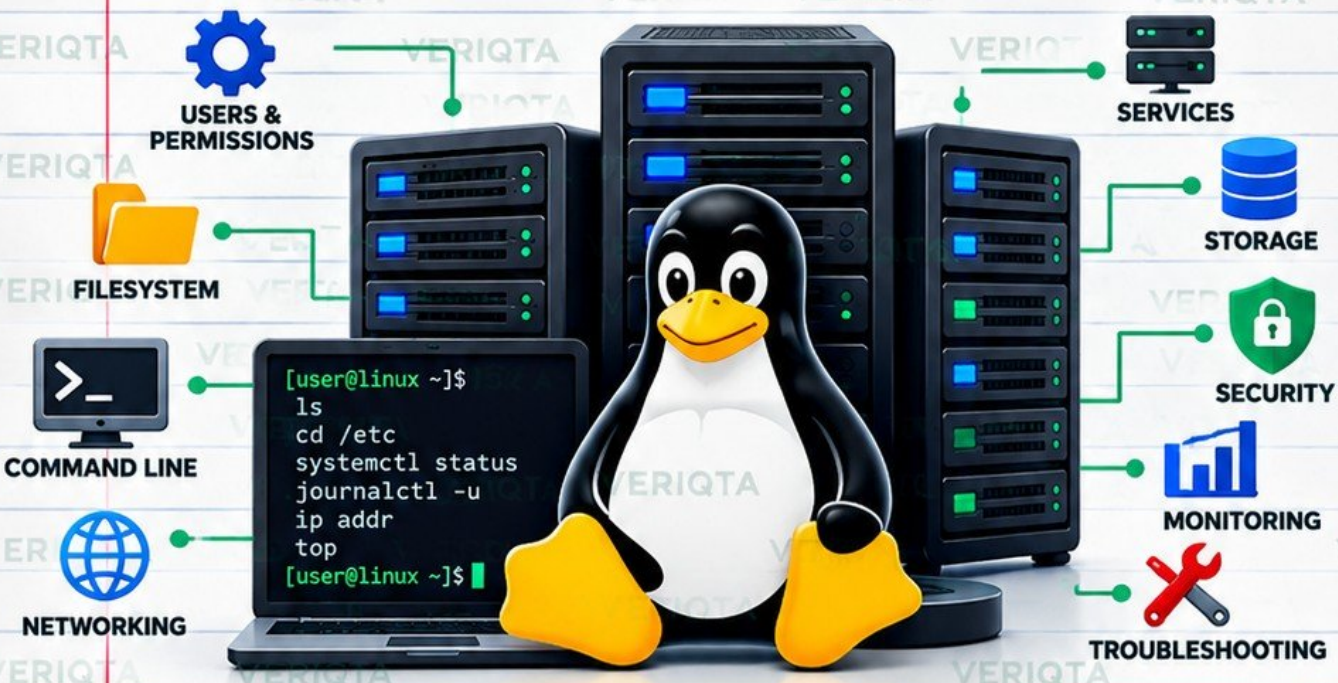
VERIQTA

Linux Essentials for Engineers



REAL EXAMPLES | PRACTICAL COMMANDS | PRODUCTION SCENARIOS | SENIOR-LEVEL INSIGHTS

@veriqta @veriqta @veriqta @veriqta @veriqta @veriqta

VERIQTA

01. LINUX ARCHITECTURE AND OPERATING SYSTEM INTERNALS

Kernel architecture, user space vs kernel space, system calls, hardware abstraction, GNU utilities, distributions and the relationship between applications and the OS.

1. What is Linux?

- Linux is an open-source operating system built around the Linux kernel.
- It provides a stable, secure and flexible platform for servers, cloud, devices and embedded systems.
- A complete Linux system includes the kernel, GNU user-space tools, system libraries and applications.

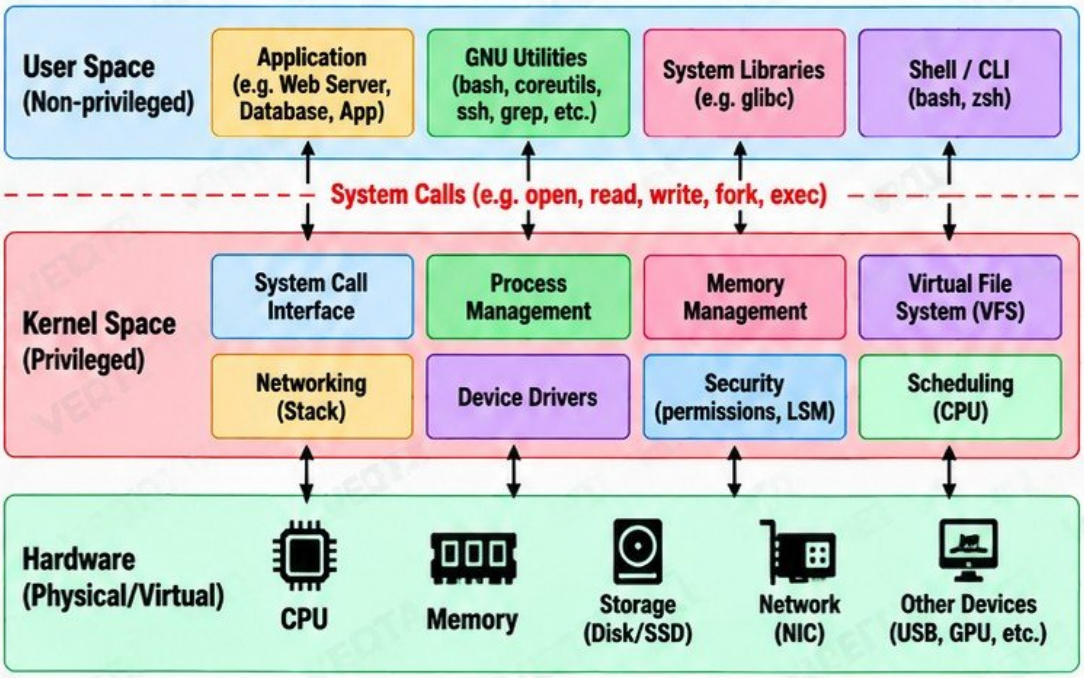
2. Key Components

- **Linux Kernel:** Manages hardware and system resources.
- **GNU Utilities:** Core user-space tools (bash, coreutils, etc).
- **System Libraries:** Provide interfaces to system calls (e.g., glibc).
- **Distributions:** Combine kernel, GNU tools, libraries and package management (e.g., Ubuntu, RHEL).

3. How It Works

- Applications run in user space and request services via system calls.
- The kernel handles the request, interacts with hardware through device drivers, and returns a response.
- The kernel abstracts hardware complexity, providing a stable interface to applications.

4. Linux Architecture Overview



USER SPACE
- Runs applications and user tools.
- Cannot access hardware directly.
- Uses system calls to request services.

KERNEL SPACE
- Runs with high privileges.
- Manages hardware and system resources.
- Provides abstraction and isolation.

HARDWARE
- Physical or virtual devices.
- Accessed via device drivers.
- Details are abstracted by the kernel.

5. GNU/Linux Distributions

- A distribution packages the kernel, GNU tools, libraries and applications together.
- Popular distributions: Ubuntu, RHEL, Debian, Fedora, AlmaLinux, CentOS, openSUSE.
- Each distribution provides its own package manager and default tools.

6. System Calls Example

Applications use system calls to request kernel services.

```
# Open a file (user space)
fd = open("/etc/hostname", O_RDONLY)

# Under the hood:
# 1. libc prepares the system call
# 2. CPU switches to kernel mode
# 3. Kernel handles the request
# 4. Data is returned to user space
```

7. Key Takeaways

- Applications do not access hardware directly.
- System calls are the bridge between user space and the kernel.
- The kernel provides hardware abstraction, resource management and security.
- GNU utilities, libraries and distributions complete the Linux ecosystem.

VERIQTA

02. LINUX FILESYSTEM HIERARCHY AND STORAGE ORGANIZATION

Filesystem Hierarchy Standard (FHS), /etc, /var, /proc, /sys, /dev, /run, mount points, inodes, symbolic links, and filesystem organizatiion in production environments.

1. What is the Filesystem Hierarchy?

- Linux uses a structured directory layout defined by the Filesystem Hierarchy Standard (FHS).
- Each directory has a specific purpose.
- It keeps the system organized, predictable and easier to manage in production.

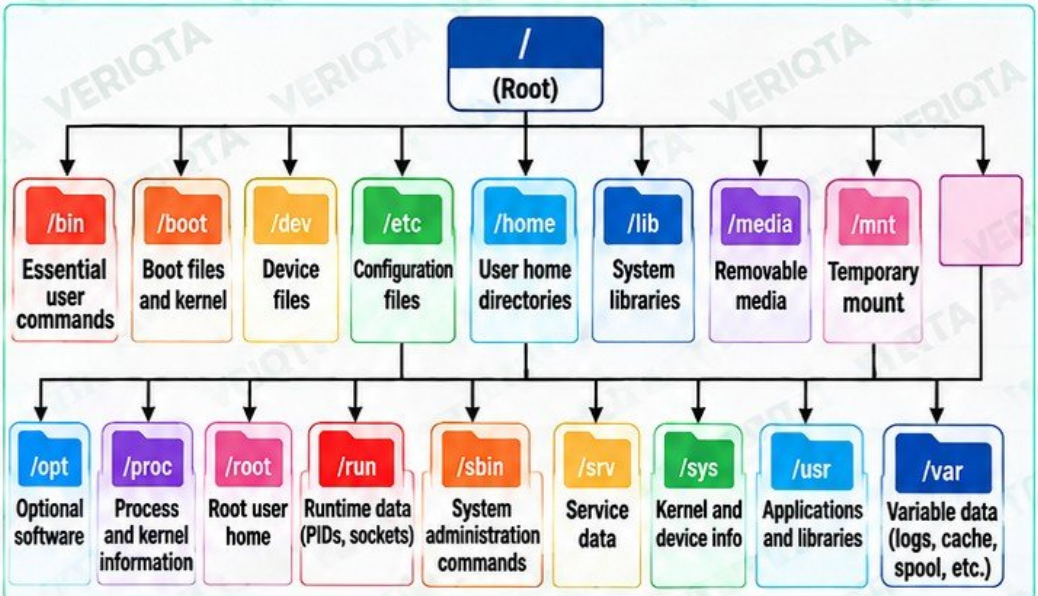
2. Key Directories and Their Purpose

- / — Root of the filesystem.
- /etc — System configuration files.
- /var — Variable data (logs, spool, cache).
- /proc — Virtual filesystem for kernel and process information.
- /sys — Kernel device and subsystem information.
- /dev — Device files.
- /run — Runtime data (PIDs, sockets).
- /home — User home directories.
- /usr — User applications and libraries.

3. Production Importance

- Proper filesystem organization improves security, stability and maintainability.
- Helps with backups and recovery.
- Allows correct monitoring and capacity planning.
- Misplacing files can break applications and services.

4. Linux Filesystem Hierarchy (FHS) Overview



5. Special Filesystems

/proc
- Virtual filesystem.
- Shows running processes and kernel information.
- Not stored on disk.

/sys
- Exposes kernel device and subsystem info.
- Used for system and device management.
- Not stored on disk.

/dev
- Device files for hardware and virtual devices.
- Managed by udev.

/run
- Temporary runtime data (PIDs, sockets, state).
- Cleared on reboot.

6. Mount Points and Storage Organization

- Linux filesystems can be on different partitions, disks or network storage.
- Common production layout:

Mount Point	Purpose	Example Device
/	Root filesystem	/dev/sda1
/boot	Boot files	/dev/sda2
/var	Logs and variable data	/dev/sdb1
/home	User data	/dev/sdb2
/data	Application data	/dev/sdc1
/mnt	Temporary mounts	NFS/EBS/Other

7. Inodes and Symboli Links

Inodes
- Store file metadata (permissions, owner, size, timestamps).
- Files are identified by inode numbers, not names.
- View inode: stat filename or ls -i filename.

Symbolic Links (Symlinks)
- A link that points to another file or directory.
- Useful for flexible file locations and application deployments.
- Create: ln -s /path/target linkname

8. Example Commands

Task	Command
View filesystem usage	df -h
List mounted filesystems	mount
View inode details	ls -i file.txt
Show file metadata	stat file.txt
Find mount points	findmnt
Create a symlink	ln -s target link
View /proc information	ls /proc
View /sys devices	ls /sys

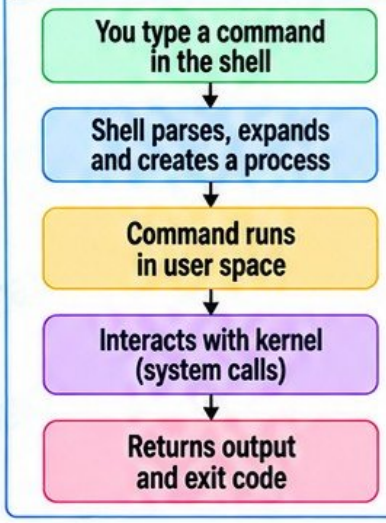
03. ADVANCED LINUX COMMAND-LINE OPERATIONS

Shell execution, command chaining, pipelines, input/output redirection, standard streams, exit codes, quoting, command substitution and safe command execution.

1. How Commands Are Executed

- You type a command in the shell (e.g., bash).
- Shell parses the command, expands variables and handles quoting.
- It creates a process and executes the command.
- The command interacts with the OS and returns an exit code.

2. Command Execution Flow



3. Standard Streams

Stream	Number	Purpose
stdin	0	Input to the command (default: keyboard)
stdout	1	Standard output (default: terminal)
stderr	2	Error output (default: terminal)

4. Input / Output Redirection

```
# Redirect output to a file
ls > files.txt
# Append output
ls >> files.txt
# Redirect errors
ls 2> errors.log
# Redirect both stdout and stderr
ls > output.log 2>&1
# Read input from a file
sort < names.txt
```

5. Command Chaining

```
# Run commands in sequence
mkdir app && cd app && touch file.txt
# Run next command only if previous succeeds
make build && make test
# Run next command even if previous fails
make build ; make test
```

6. Pipelines

```
# Send output of one command to another
ps aux | grep nginx
# Multiple stages
cat access.log | grep "/api" |
  awk '{print $1}' | sort | uniq -c
# Count running processes
ps aux | wc -l
```

7. Exit Codes

- Every command returns an exit code.
- 0 usually means success.
- Non-zero means an error occurred.
- You can check it using $?.

```
# Successful command
ls
echo $?          # Output: 0

# Failed command
ls /notfound
echo $?          # Output: 2
```

8. Quoting

- Single quotes ' ' prevent variable expansion.
- Double quotes " " allow variable expansion.
- Use quotes to handle spaces and special characters.

```
# Single quotes (no expansion)
echo '$HOME"
# Double quotes (expansion)
echo "$HOME"
# Handle spacies in filenames
cat "my file.txt"
```

9. Command Substitution

- Use $(...) or backticks `...` to run a command and use its output.
- Common in scripts and automation.

```
# Using $(...)
today=$(date +%F))
echo "Today is $today"

# Usndle backticks (older syntax)
count=`ls | wc -l`
echo "File count: $count"
```

10. Safe Command Execution

- Always check the command before running it.
- Use --dry-run or --verbose when available.
- Be careful with destructive commands (rm, mv, dd).
- Use quotes to prevent unexpected expansion.
- Validate user input in scripts.
- Prefer absolute paths for critical operations.

11. Useful Advanced Examples

```
# Find large files and show top 5
du -ah /var | sort -rh | head -5
# Find processes using a port
lsof -i :8080
# Extract specific fields
ps aux | awk '{print $1, $11}'
# Search and exclude pattern
grep -v "health" access.log | tail -20
```

12. Common Pitfalls

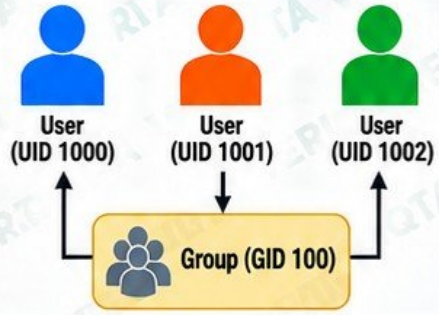
- Forgetting to quote variables.
- Accidentally overwriting files instead of appending.
- Not checking exit codes.
- Using relative paths in scripts.
- Running commands with unnecessary root privileges.
- Piping sensitive data to logs.

04. LINUX USERS, GROUPS, PERMISSIONS, AND ACCESS CONTROL

User and group identities, ownership, permission bits, chmod, chown, umask, setuid, setgid, sticky bit, access control lists (ACLs), and least-privilege administration.

1. Users and Groups

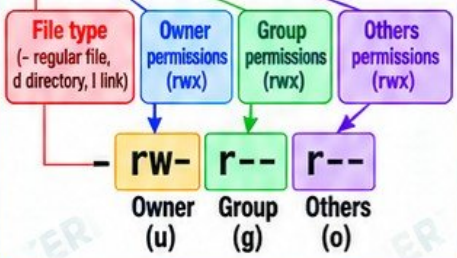
- Every user has a unique user ID (UID).
- Every group has a unique group ID (GID).
- Users and groups are stored in /etc/passwd and /etc/group.
- Groups help manage access to files and system resources.



2. File Ownership and Permissions

- Each file has an owner, a group and permission bits.
- Permissions control who can read (r), write (w) or execute (x) the file.
- Use ls -l to view permissions.

```
$ ls -l /etc/passwd
-rw-r--r-- 1 root root 1740 Jul 10 10:20 /etc/passwd
```



3. Permission Bits

Symbol	Value	Meaning
r	4	Read (view file)
w	2	Write (modify file)
x	1	Execute (run file)

Common chmod Examples

```
chmod 755 script.sh  # rwxr-xr-x
chmod 644 file.txt   # rw-r--r--
chmod 700 private.sh # rwx------
```

Numeric Representation

```
755 = rwx r-x r-x
644 = rw- r-- r--
```

4. Changing Ownership

Use chown to change the owner and group of a file or directory.

```
# Change owner
sudo chown user1 file.txt
# Change group
sudo chown :devops file.txt
# Change owner and group
sudo chown user1:devops file.txt

# Recursive change on a directory
sudo chown -R user1:devops /app
```

5. The umask

- umask sets the default permissions for new files and directories.
- It subtracts permissions from a default value (666 for files, 777 for directories).
- Common umask values: 022, 027.

```
$ umask
0022
$ touch newfile.txt
$ ls -l newfile.txt
-rw-r--r-- 1 user user 0 Jul 10 11:00 newfile.txt
```

6. Special Permission Bits

- **setuid (4):** Run a program with the file owner's permissions.
- **setgid (2):** Run a program with the group's permissions or inherit group on directories.
- **sticky bit (1):** On directories, only the file owner can delete their own files (e.g., /tmp).

Permission	Symbol	Example	Meaning
setuid	SUID	chmod 4755 file	Run as file owner
setgid	SGID	chmod 2755 dir	Run as group / inherit group
sticky bit	Sticky	chmod 1777 dir	Only owner can delete files

7. Access Control Lists (ACLs)

- ACLs provide more fine-grained permissions than the traditional owner/group/others model.
- Use setfacl and getfacl to manage ACLs.

```
# Set an ACL: give user2 read access
sudo setfacl -m u:user2:r file.txt
# View ACLs
getfacl file.txt
# Remove ACL
sudo setfacl -x u:user2 file.txt
```

8. Least-Privilege Administration

- Give users only the permissions they need.
- Use groups instead orf giving individual permissions.
- Avoid running applications as root.
- Use sudo for administrative tasks.
- Review permissions regularly.
- Apply ACLs when detailed control is required.

9. Useful Commands

id	Show user and group IDs
whoami	Show current user
groups	Show user's groups
ls -l	View file permissions
chmod	Change permissions
chown	Change owner/group
umask	Show or set default permissions
setfacl	Set access control list
getfacl	View access control list
find	Find files by permissions `find / -perm 644`

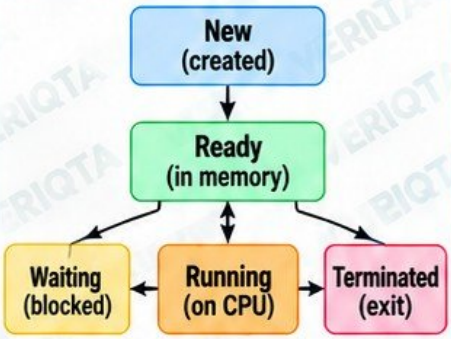
05. LINUX PROCESSES, SIGNALS, AND JOB CONTROL

Process lifecycle, process identifiers, parent-child relationships, process states, signals, foreground/background jobs, zombie processes, and process investigation using ps, top, and kill.

1. What is a Process?

- A process is a running instance of a program.
- Each process has a unique Process ID (PID).
- Processes use system resources (CPU, memory, files, network).
- A process can create child processes.
- When a process ends, it returns an exit status code.

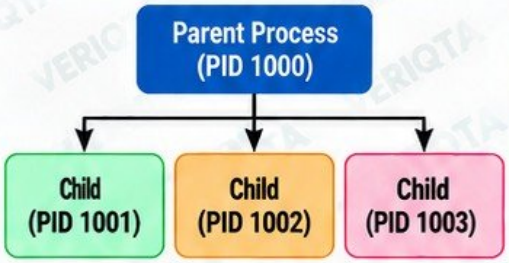
2. Process Lifecycle



3. Process States

State	Code	Description
Running	R	Currently executing or ready to run
Sleeping	S	Waiting for an event (e.g., I/O)
Uninterruptible	D	Waiting for I/O (cannot be interrupted)
Stopped	T	Stopped (e.g., by signal)
Zombie	Z	Process finished but still in process table (waiting for parent)

4. Parent-Child Relationship



- Every process (except init) has a parent.
- Child processes inherit resources.
- When a parent exits, children are reparented to init (PID 1).

5. Process Signals

- Signals are used to notify a process to perform an action.
- Common signals:

Signal	Number	Purpose
SIGTERM	15	Graceful termination (default for kill)
SIGKILL	9	Force termination (cannot be ignored)
SIGHUP	1	Reload configuration
SIGINT	2	Interrupt (Ctrl+C)
SIGTSTP	20	Stop (Ctrl+Z)
SIGCONT	18	Continue a stopped process

6. Job Control

- Run jobs in foreground (current terminal) or background.
- Use & to run in background.
- Use Ctrl+Z to suspend a process.
- Use jobs, bg, and fg to manage jobs.

```
# Run a command in background
sleep 300 &
[1] 1234
# View jobs
jobs
# Bring job to foreground
fg %1
# Send job to background
bg %1
```

7. Zombie Processes

- A zombie process has finished execution but its parent has not collected its exit status.
- Zombies remain in the process table with state 'Z'.
- They do not use CPU or memory but should be cleaned up by the parent.

```
$ ps -ef | grep defunct
user   4321   1 0  10:20 ?    00:00:00 [myapp] <defunct>
```

8. Process Investigation with ps

- The ps command shows running processes.
- Common options:

```
# Show all processes
ps aux
# Show processes in tree view
ps -ef --forest
# Find a specific process
ps aux | grep nginx
# Show process by PIDD
ps -p 1234 -o pid,ppid,cmd,%mem,%cpu
```

9. Using top and htop

- top shows real-time process activity.
- Press q to quit.
- htop (if installed) provides a more interactive view.

```
# Start top
top
# Or use htop (if available)
htop
```

10. Killing Processes

- Use kill with a signal to terminate a process.
- Use SIGTERM first, then SIGKILL if needed.

```
# Graceful termination
kill 1234       # sends SIGTERM
# Force termination
kill -9 1234    # sends SIGKILL
```

11. Useful Commands

Task	Command
View all processes	ps aux
View process tree	ps -ef --forest
Find a process	pgrep nginx
Show open files (by PID)	lsof -p 1234
Show process resources	top or htop
Send a signal	kill -15 1234
Force kill	kill -9 1234

12. Key Takeaways

- Every process has a unique PID.
- Processes go through different states (running, sleeping, stopped, zombie).
- Signals control process behavior.
- Use job control to manage foreground and background tasks.
- Investigate processes with ps, top, and other tools.
- Avoid leaving zombie processes.
- Always use graceful termination before force killing.

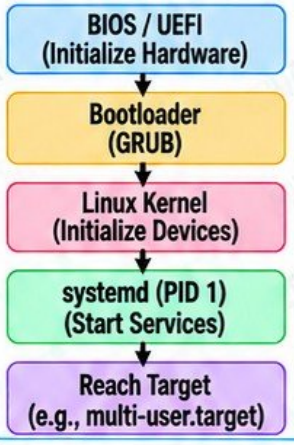
06. SYSTEMD, SERVICES, AND LINUX BOOT MANAGEMENT

Linux boot sequence, systemd units, service dependencies, targets, systemctl, journalctl, service restart policies, and troubleshooting failed services.

1. Linux Boot Sequence

- BIOS/UEFI initializes hardware.
- Bootloader (e.g., GRUB) loads Linux kernel.
- Kernel initializes devices and mounts the root filesystem.
- systemd (PID 1) starts and manages services.
- Targets are reached (e.g., multi-user).
- System is ready for user logins and applications.

2. Boot Process Flow



3. Key systemd Concepts

- **Unit**: A configuration file that describes a resource (service, socket, mount, etc).
- **Service**: A unit that runs a long-running process or a one-time task.
- **Target**: A group of units, similar to a runlevel.
- **Dependency**: Defines the order in which units start.
- **systemctl**: Command to manage units.
- **journalctl**: Command to view logs from systemd and services.
- **Restart Policy**: Defines what to do if a service fails.

4. Common Unit Types

Unit Type	Purpose
service	Runs a process or application
target	Groups units (like runlevels)
socket	Listens for incoming connections
mount	Mounts a filesystem
timer	Runs a service on a schedule
path	Starts a service when a file changes
device	Represents a hardware device
slice	Groups services for resource control
scope	Manages externally created processes

5. Common Targets

Target	Description
default.target	Default target (usually multi-user)
multi-user.target	Multi-user mode (no GUI)
graphical.target	Multi-user mode with GUI
rescue.target	Minimal system for recovery
emergency.target	Very limited shell (for maintenance)

6. Essential systemctl Commands

Task	Command
Start a service	`systemctl start nginx`
Stop a service	`systemctl stop nginx`
Restart a service	`systemctl restart nginx`
Enable at boot	`systemctl enable nginx`
Disable at boot	`systemctl disable nginx`
Check status	`systemctl status nginx`
List all services	`systemctl list-units --type=service`
View dependencies	`systemctl list-dependencies nginx`
View current target	`systemctl get-default`
Change target	`systemctl isolate multi-user.target`

7. Example Service Unit File

```
[Unit]
Description=My Custom Service
After=network.target

[Service]
ExecStart=/usr/local/bin/myapp
Restart=on-failure
RestartSec=5
User=appuser
Group=appgroup

[Install]
WantedBy=multi-user.target
```

Metadata and dependencies

Service settings and restart policy

Which target should start this service

8. Restart Policies

Policy	Description
no	Do not restart (default)
on-success	Restart if service exits successfully
on-failure	Restart if service exits with failure
on-abnormal	Restart on abnormal termination
always	Always restart
unless-stopped	Always restart unless manually stopped

9. Viewing Logs with journalctl

```
# View all logs
journalctl
# Follow logs in real time
journalctl -f
# View logs for a specific service
journalctl -u nginx
# Show logs since recent time
journalctl -u nginx --since "1 hour ago"
# Show only errors
journalctl -u nginx -p err
```

10. Troubleshooting Failed Services

- Check service status: `systemctl status <service>`
- View logs: `journalctl -u <service>`
- Check configuration files (e.g., `/etc/<service>/`).
- Verify dependencies: `systemctl list-dependencies <service>`
- Look for common issues: missing files, permission errors, or port conflicts.
- Use systemd-analyze to check boot time and service delays.

11. Key Takeaways

- systemd is the first process (PID 1) and manages the entire system.
- Units, services and targets define how the system is started and managed.
- systemctl is used to control and inspect services.
- journalctl provides detailed logs for troubleshooting.
- Restart policies improve service reliability.
- Understanding dependencies helps you fix startup and runtime issues.

07. LINUX NETWORKING FUNDAMENTALS AND DIAGNOSTICS

Network interfaces, IP addressing, routing tables, DNS resolution, TCP/UDP, listening sockets, and troubleshooting with ip, ss, dig, ping, and traceroute.

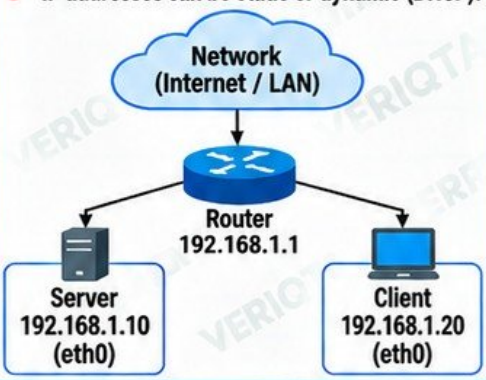
1. Network Interfaces

- A network interface connects your Linux system to a network.
- Interfaces can be physical (e.g., eth0) or virtual (e.g., lo, ens33, docker0).
- Each interface has an IP address, MAC address and status (up/down).

```
# Show network interfaces
ip link show
# Show IP addresses
ip addr show
# Bring an interface up/down
sudo ip link set eth0 up
sudo ip link set eth0 down
```

2. IP Addressing

- IPv4 uses 32-bit addresses (e.g., 192.168.1.10).
- IPv6 uses 128-bit addresses.
- Each interface can have multiple IP addresses.
- IP addresses can be static or dynamic (DHCP).



3. Routing Tables

- The routing table determines where to send network traffic.
- Each route has a destination, gateway and interface.
- Use ip route to view and manage routes.

```
# View routing table
ip route show
# Add a route (example)
sudo ip route add 10.0.0.0/24
via 192.168.1.1
# Default route
ip route show default
```

4. DNS Resolution

- DNS converts domain names to IP addresses.
- Linux uses /etc/resolv.conf for DNS servers.
- Use dig, nslookup or host to query DNS.

```
# Query a domain
dig veriqta.com
# Show DNS servers
cat /etc/resolv.conf
# Reverse DNS lookup
dig -x 8.8.8.8
```

5. TCP and UDP

- TCP: Connection-oriented, reliable, used for web, SSH, databases, etc.
- UDP: Connectionless, faster, used for DNS, streaming, monitoring, etc.

Feature	TCP	UDP
Connection	Yes	No
Reliability	Reliable	Best effort
Speed	Slower	Faster
Use cases	HTTP, SSH, Databases	DNS, DHCP, Streaming
Acknowledgement	Yes	No
Data delivery	Ordered	Not guaranteed

6. Listening Sockets

- Services listen on specific ports for incoming connections.
- Use ss or netstat to view listening sockets.

```
# Show listening ports
ss -tulnp
# Show only TCP ports
ss -tlnp
# Show only UDP ports
ss -ulnp
# Using netstat (if installed)
netstat -tulnp
```

7. Network Diagnostic Commands

Command	Purpose	Example
ip	Manage interfaces, addresses and routes	ip addr show
ss	Show listening sockets and connections	ss -tulnp
ping	Test connectivity	ping 8.8.8.8
traceroute	Show network path to destination	traceroute veriqta.com
dig	Query DNS records	dig veriqta.com
nslookup	DNS lookup tool	nslookup veriqta.com
curl	Test HTTP connectivity	curl -I https://veriqta.com

8. Example Troubleshooting

Check if a host is reachable
`ping -c 4 veriqta.com`

Trace the network path
`traceroute veriqta.com`

Check DNS resolution
`dig veriqta.com`

Check listening services
`ss -tulnp | grep :80`

View interface and IP
`ip addr show`

Check routing table
`ip route show`

9. Common Issues and Fixes

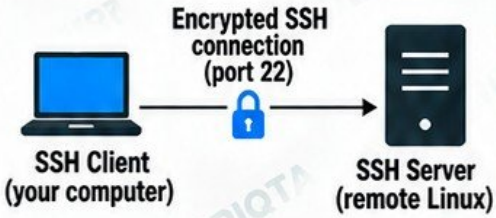
Issue	Possible Cause	Fix
No connectivity	Interface down	ip link set eth0 up
Cannot resolve domain name	DNS server issue	Check /etc/resolv.conf or use public DNS
Cannot reach remote network	Missing route	Add correct route with ip route add
Connection refused	Service not running	Check service and listening port
High latency	Network congestion or routing issue	Use traceroute and check network path
Intermittent drops	Firewall or network instability	Check firewall rules and logs

08. SSH, REMOTE ACCESS, AND SECURE SERVER ADMINISTRATION

SSH authentication, public/private key pairs, authorized_keys, SSH agent, host verification, secure configuration, bastion hosts, and diagnosing remote-access failures.

1. What is SSH?

- SSH (Secure Shell) provides encrypted remote access to Linux systems.
- It replaces insecure protocols like telnet.
- Used for administration, file transfer (SCP/SFTP) and tunneling.
- Provides authentication, confidentiality, and integrity.



2. Key Pair Authentication

- A key pair consists of a private key (kept secret) and a public key (placed on the server).
- More secure than password authentication.
- The public key is stored in ~/.ssh/authorized_keys on the server.

```
# Generate a key pair (on local machine)
ssh-keygen -t ed25519 -C "your@email.com"
# View the public key
cat ~/.ssh/id_ed25519.pub
# Copy public key to server
ssh-copy-id user@server_ip
# Or manually add to authorized_keys
cat ~/.ssh/id_ed25519.pub >>
~/.ssh/authorized_keys
```

3. authorized_keys File

- Contains public keys allowed to log in.
- Each line is a public key.
- Permissions must be correct (700 for .ssh and 600 for the file).

```
# On the server
mkdir -p ~/.ssh
chmod 700 ~/.ssh

# Add your public key
nano ~/.ssh/authorized_keys

# Set correct permissions
chmod 600 ~/.ssh/authorized_keys
```

4. Using SSH Agent

- SSH agent keeps your private key in memory so you don't have to enter the passphrase repeatedly.
- Useful when working with multiple servers.

```
# Start ssh-agent
eval "$(ssh-agent -s)"
# Add your private key
ssh-add ~/.ssh/id_ed25519
# List loaded keys
ssh-add -l
```

5. Host Verification and Known Hosts

- When connecting to a new server, SSH verifies the server's host key.
- The key is stored in ~/.ssh/known_hosts.
- Helps prevent man-in-the-middle attacks.

```
# First connection (shows fingerprint)
ssh user@server_ip
# View known hosts
cat ~/.ssh/known_hosts
# Remove an old host entry
ssh-keygen -R server_ip
```

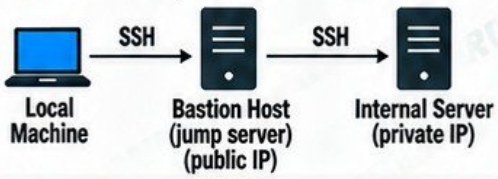
6. Secure SSH Configuration

- Disable password authentication (use key-based auth).
- Change default port if needed.
- Restrict users, use SSH key only.
- Configure idle timeouts and logging.

```
# /etc/ssh/sshd_config (key settings)
Port 22
PermitRootLogin no
PasswordAuthentication no
AllowUsers devops admin
ClientAliveInterval 300
ClientAliveCountMax 2
LogLevel VERBOSE
```

7. Bastion Hosts (Jump Servers)

- A bastion host is a secure gateway server used to access internal servers.
- Improves security by limiting direct access.
- Common in production environments.




```
# Connect to internal server through bastion
ssh -J user@bastion_ip user@internal_ip
# Or using ProxyJump in config file
Host internal
  HostName internal_ip
  User user
  ProxyJump user@bastion_ip
```

8. Diagnosing Remote-Access Failures

- Check network connectivity.
- Verify SSH service is running.
- Check firewall rules.
- Review logs on client and server.
- Confirm correct user, key and permissions.

```
# Test network connectivity
ping server_ip
# Check if SSH port is open
nc -zv server_ip 22
# Verbose SSH connection (debug)
ssh -v user@server_ip
# Check SSH service on server
sudo systemctl status sshd
# Check logs on server
sudo journalctl -u sshd -n 50
# Check firewall (UFW example)
sudo ufw status
```

9. Common Issues and Fixes

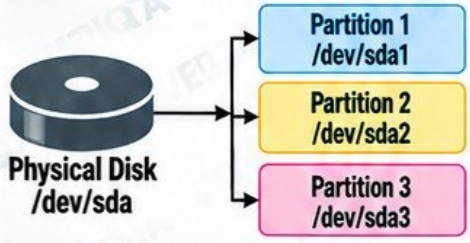
Issue	Possible Cause	Fix
Connection timed out	Server unreachable or firewall blocking	Check IP, network and firewall rules (ufw, security group).
Permission denied (publickey)	Key not added or wrong permissions	Verify key in authorized_keys and correct permissions (700/600).
Host key verification failed	Server key changed	Remove old key: ssh-keygen -R server_ip
Connection refused	SSH service not running	sudo systemctl start sshd
Too many authentication failures	Too many keys trying to authenticate	Use ssh -o IdentitiesOnly=yes
Access denied for user	User not allowed in sshd_config	Check AllowUsers and user existence.

09. LINUX STORAGE, PARTITIONS, AND FILESYSTEM MANAGEMENT

Block devices, partitions, filesystems, logical volume management (LVM), mounting, /etc/fstab, disk utilization, inode exhaustion, and storage expansion.

1. Storage Fundamentals

- Linux stores data on block devices such as HDDs, SSDs, and virtual disks.
- Devices appear as /dev/sdX, /dev/nvmeX or /dev/mapper/* (LVM).
- A disk can be divided into partitions and formatted with a filesystem (e.g., ext4, xfs).
- Filesystems organize data, metadata and inodes.



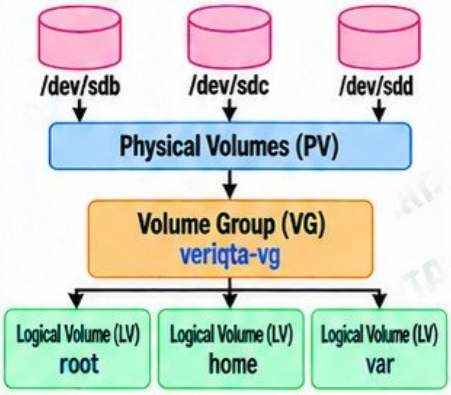
2. Partitions and Filesystems

- Use partitioning tools like fdisk, parted or gdisk.
- Common filesystems: ext4, xfs, btrfs.
- Create, view and manage partitions and filesystems.

```
# View block devices
lsblk
# View partition table
sudo fdisk -l
# Create a new partition (interactive)
sudo fdisk /dev/sdb
# Create filesystems
sudo mkfs.ext4 /dev/sdb1
# View filesystem type
sudo blkid /dev/sdb1
```

3. Logical Volume Management (LVM)

- LVM allows flexible storage management.
- Components: Physical Volume (PV), Volume Group (VG), Logical Volume (LV).
- Easy to extend, reduce and manage storage without downtime.



4. Mounting Filesystems

- A filesystem must be mounted to a directory to access its data.
- Mount points are directories (e.g., /mnt/data).
- Use mount and umount to manage mounts.

```
# Create a mount point
sudo mkdir -p /mnt/data
# Mount a filesystem
sudo mount /dev/sdb1 /mnt/data
# Unmount a filesystem
sudo umount /mnt/data
```

5. /etc/fstab Configuration

- /etc/fstab defines filesystems to mount automatically at boot.
- Use UUID or device names for reliable mounting.
- Include filesystem type and options (e.g., defaults, noatime).

```
# Example /etc/fstab entries
UUID=123e4567-e89b-12d3-a456-426614174000
/ext4          defaults                0 2

UUID=7c9a2d1f-3b4e-5c6d-7f8a-90b1c2d3e4f5
/mnt/data      xfs      defaults       0 2
```

6. Disk Utilization and Inodes

- Monitor disk space usage with df.
- Check inode usage with df -i.
- High disk usage or inode exhaustion can cause application failures.

```
# Check disk usage
df -h
# Check inode usage
df -i
# Detailed usage per directory
du -sh /var/*
# Find large directories
du -h --max-depth=1 / | sort -hr
```

7. Storage Expansion (Examples)

- Expand an LVM logical volume.
- Grow the filesystem to use the new space.
- Works without downtime for many filesystems (e.g., xfs, ext4).

```
# Extend logical volume by 10G
sudo lvextend -L +10G /dev/veriqta-vg/home
# For ext4
sudo resize2fs /dev/veriqta-vg/home
# For xfs
sudo xfs_growfs /home
```

8. Useful Commands

Command	Purpose
lsblk	List block devices and partitions
fdisk / parted	Create and manage partitions
mkfs.ext4/xfs	Create filesystem
blkid	Show filesystem UUID and type
mount / umount	Mount or unmount filesystem
df -h	Show disk space usage
df -i	Show inode usage
du -sh	Show directory size
lvcreate/lvextend	Create/extend LVM logical volume
resize2fs / xfs_growfs	Grow filesystem
ls -l /dev/disk/by-uuid	View UUID device links

9. Key Takeaways

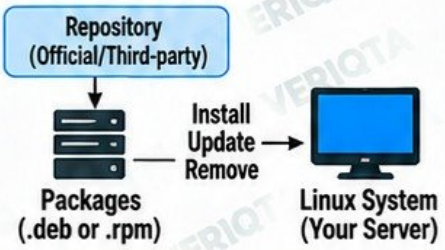
- Storage is accessed through block devices, partitions and filesystems.
- LVM provides flexible and scalable storage management.
- /etc/fstab ensures filesystems mount automatically at boot.
- Monitor both disk space and inode usage.
- Plan storage expansion to avoid service disruption.
- Use appropriate filesystem options for production environments.

10. LINUX PACKAGE MANAGEMENT AND SOFTWARE LIFECYCLE

APT, DNF, repositories, package dependencies, version pinning, security updates, package verification, patch management, and controlled software upgrades.

1. Package Management Overview

- Package managers automate the installation, update, configuration and removal of software.
- Handle dependencies automatically.
- Use repositories (official or third-party) to obtain packages.
- Keep the system secure with updates and patches.



2. APT vs DNF

Feature	APT (Debian/Ubuntu)	DNF (RHEL/CentOS/Fedora)
Package format	.deb	.rpm
Main command	apt / apt-get	dnf
Config files	/etc/apt/	/etc/dnf/
Repository definition	sources.list	.repo files
Update command	apt update	dnf check-update
Upgrade command	apt upgrade	dnf upgrade
Install package	apt install	dnf install
Remove package	apt remove	dnf remove
Search package	apt search	dnf search
Show package info	apt show	dnf info

3. Common Package Commands

```
# APT (Ubuntu/Debian)
sudo apt update              # Update repo index
sudo apt upgrade             # Upgrade packages
sudo apt install nginx       # Install package
sudo apt remove nginx        # Remove package
sudo apt search nginx        # Search package
sudo apt show nginx          # Show package info
sudo apt list --upgradable   # List available updates

# DNF (RHEL/CentOS/Fedora)
sudo dnf check-update        # Check for updates
sudo dnf upgrade             # Upgrade packages
sudo dnf install nginx       # Install package
sudo dnf remove nginx        # Remove package
sudo dnf search nginx        # Search package
sudo dnf info nginx          # Show package info
sudo dnf list updates        # List available updates
```

4. Repositories

- Repositories store packages and metadata.
- Use official repositories for stability.
- Third-party repositories provide additional software.
- Keep repositories updated to get the latest security fixes.

```
# Ubuntu/Debian: view repositories
cat /etc/apt/sources.list
cat /etc/apt/sources.list.d/*.list

# RHEL/CentOS/Fedora: view repositories
dnf repolist
cat /etc/yum.repos.d/*.repo
```

5. Package Dependencies

- Packages often depend on other packages.
- The package manager resolves and installs dependencies automatically.
- Use commands to check dependencies.

```
# Check dependencies (APT)
apt show nginx | grep Depends
apt depends nginx

# Check dependencies (DNF)
dnf repoquery --requires nginx
dnf repoquery --whatrequires nginx
```

6. Version Pinning

- Version pinning prevents automatic upgrades to a specific version.
- Useful for stability in production environments.

```
# APT: install a specific version
sudo apt install nginx=1.24.0-1
# Hold a package (prevent upgrades)
sudo apt-mark hold nginx
# DNF: install a specific version
sudo dnf install nginx-1.24.0
# Exclude a package from upgrades
# In /etc/dnf/dnf.conf
[main]
exclude=nginx
```

7. Security Updates and Patching

- Regularly update packages to fix vulnerabilities.
- Check for security advisories.
- Apply patches in a controlled manner.

```
# Ubuntu/Debian
sudo apt update
sudo apt upgrade
sudo unattended-upgrades  # Auto updates

#  RHEL/CentOS/Fedora
sudo dnf update
sudo dnf upgrade
sudo dnf update --security # Security only
```

8. Package Verification

- Verify package integrity using checksums or GPG signatures.
- Ensures packages are not tampered with.

```
# Check package file (RPM)
rpm -K package.rpm
# Check package file (DEB)
debsums -c nginx
# Import and verify GPG key
sudo rpm --import RPM-GPG-KEY
# Verify repository signatures)
sudo apt-key list  # (legacy)
# or use signed-by in sources.list
```

9. Controlled Software Upgrades

- Test upgrades in non-production first.
- Review release notes.
- Use version pinning or exclude critical packages if needed.
- Have a rollback plan (e.g., snapshots).

```
# Upgrade all packages
sudo apt upgrade   # or: sudo dnf upgrade
# Upgrade a specific package
sudo apt install nginx # or: sudo dnf upgrade nginx
# Simulate upgrade (DNF)
sudo dnf upgrade --assumeno
# Downgrade package (RHEL/CentOS)
sudo dnf downgrade nginx
# Use snapshots (if available)
# e.g., LVM, filesystem snapshots or VM snapshots
```

10. Key Takeaways

- Use package managers (APT or DNF) to manage software.
- Keep repositories updated and use trusted sources.
- Understand and manage dependencies.
- Use version pinning for critical software.
- Regularly apply security updates and patches.
- Always verify package integrity.
- Plan and test upgrades in a controlled manner.

11. LINUX SHELL SCRIPTING AND OPERATIONAL AUTOMATION

Bash variables, conditionals, loops, functions, argument handling, exit statuses, error handling, set -euo pipefail limitations, and safe automation patterns.

1. Bash Script Structure

- Use a shebang to specify the shell.
- Keep scripts readable and modular.
- Use meaningful variable and function names.
- Add comments and proper error handling.

```
#!/bin/bash
# Example script structure
set -euo pipefail

echo "Starting script..."
# commands here
echo "Script completed."
```

2. Variables and Input

- Store and use data with variables.
- Quote variables to prevent word splitting.
- Read user input and script arguments.

```
#!/bin/bash
name="Veriqta"
echo "Hello, $name"

read -p "Enter your name: " user
echo "Welcome, $user"

# Script arguments
echo "First argument: $1"
echo "All arguments: $@"
```

3. Conditionals

- Use if, elif, else for decision making.
- Test files, strings, numbers and commands.

```
#!/bin/bash
if [ -f /etc/passwd ]; then
    echo "File exists"
elif [ -d /etc/passwd ]; then
    echo "It is a directory"
else
    echo "File not found"
fi

# Check command success
if command -v nginx &>/dev/null; then
    echo "nginx is installed"
else
    echo "nginx is not installed"
fi
```

4. Loops

- Use for, while and until loops.
- Process files, lists and command output.

```
# For loop
for i in 1 2 3 4 5; do
    echo "Number: $i"
done

# While loop
count=1
while [ $count -le 3 ]; do
    echo "Count: $count"
    ((count++))
done

# Loop through files
for file in /var/log/*.log; do
    echo "Processing: $file"
done
```

5. Functions

- Group commands into reusable functions.
- Pass arguments to functions.
- Return values using exit status.

```
# Function definition
greet() {
    echo "Hello, $1"
}
greet "Veriqta"

add() {
   local a=$1
   local b=$2
   echo $((a + b))
}

result=$(add 5 3)
echo "Result: $result"
```

6. Exit Status and Error Handling

- Check command exit codes with $?.
- Handle errors gracefully.
- Use traps for cleanup.

```
#!/bin/bash
set -euo pipefail

trap 'echo "Error on line $LINENO"; exit 1' ERR

echo "Running command..."
if ! ls /nonexistent 2>/dev/null; then
    echo "Command failed, handling gracefully"
fi

# Manual exit status check
some_command
if [ $? -ne 0 ]; then
    echo "Command failed with exit code $?"
    exit 1
fi
```

7. set -euo pipefail Considerations

- Helps catch errors but has limitations.
- Does not handle all failure scenarios.
- Understand when to use and when to avoid it.

```
#!/bin/bash
set -euo pipefail

# Example pitfall: command in a pipeline
false | true  # set -e does not catch this
echo "Pipeline did not fail"

# Safer approach
if ! false | true; then
    echo "Pipeline failed (caught manually)"
fi
```

8. Safe Automation Patterns

- Use absolute paths.
- Validate input and check prerequisites.
- Use logging and dry-run modes.
- Make scripts idempotent and secure.

```
#!/bin/bash
set -euo pipefail
LOGFILE="/var/log/veriqta_script.log"
echo "[$(date)] Starting..." | tee -a $LOGFILE
# Dry run mode
DRY_RUN=${1:-false}
if [ "$DRY_RUN" = "true" ]; then
    echo "Dry run mode - no changes will be made."
fi
# Check prerequisite
if ! command -v nginx &>/dev/null; then
    echo "nginx is required. Exiting." | tee -a $LOGFILE
    exit 1
```

9. Common Automation Tasks

- Backup files.
- Rotate logs.
- Monitor services.
- Send alerts.

```
# Backup example
tar -czf /backup/home_$(date +%F).tar.gz /home

# Log rotation (simple example)
find /var/log -name "*.log" -mtime +7 -delete

# Check service and restart if needed
if ! systemctl is-active --quiet nginx; then
    systemctl restart nginx
    echo "nginx restarted"
fi

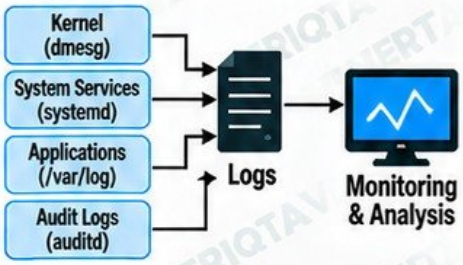
# Send alert (example using echo)
echo "Disk usage high!" | mail -s "Alert" \
    admin@veriqta.com
```

12. LINUX LOGS, JOURNALING, AND SYSTEM OBSERVABILITY

System logs, journald, log rotation, kernel messages, structured logging, resource metrics, service diagnostics, and investigating production incidents.

1. Linux Logging Overview

- Linux records events in system logs.
- Logs help with troubleshooting, monitoring and security.
- Common log sources: kernel, system services, applications and audit logs.
- Different distributions use traditional log files and/or systemd journal.



2. Traditional Log Files

- Stored in /var/log/.
- Managed by applications and syslog.
- Common logs: /var/log/messages, /var/log/syslog, /var/log/auth.log, /var/log/secure.

```
# View system log (Ubuntu/Debian)
sudo tail -n 50 /var/log/syslog

# View system log (RHEL/CentOS)
sudo tail -n 50 /var/log/messages

# View authentication logs
sudo tail -n 50 /var/log/auth.log

# View last 100 lines
tail -n 100 /var/log/secure
```

3. systemd Journald

- systemd uses journald to collect and store logs.
- Logs can be viewed with journalctl.
- Supports structured logging with metadata.
- Useful filters by unit, time, priority, etc.

```
# View recent logs
journalctl -xe
# View logs for a specific service
journalctl -u nginx
# Follow logs in real time
journalctl -u nginx -f
# View logs since a specific time
journalctl --since "2024-01-01 10:00"
# Show logs with priority (error and above)
journalctl -p err
```

4. Log Rotation

- Prevents log files from growing indefinitely.
- Controlled by logrotate.
- Keeps old logs compressed and manages retention.

```
# View logrotate configuration
cat /etc/logrotate.conf
# View config for a specific application
cat /etc/logrotate.d/nginx
# Force log rotation (test)
sudo logrotate -f /etc/logrotate.conf
# Check logrotate status
sudo logrotate -d /etc/logrotate.conf
```

5. Kernel Messages

- Kernel logs record hardware, driver and system events.
- Useful for boot issues and hardware problems.
- Viewed with dmesg or from journalctl.

```
# View recent kernel messages
dmesg | tail -n 50
# Filter kernel messages
dmesg | grep -i error
# View kernel logs from journalctl
journalctl -k
# Follow kernel logs in real time
journalctl -k -f
```

6. Resource Metrics and Observability

- Monitor CPU, memory, disk, network and processes.
- Use tools like top, htop, vmstat, iostat, free, df, ss, and sar.
- Helps identify performance issues and resource bottlenecks.

```
# CPU and memory usage
top   # or htop
# Memory usage
free -h
# Disk usage
df -h   # disk space
du -sh /var/*   # directory size
# Disk I/O                    #. app
iostat -xz 1     connections  |  ss -tulnp
# System activity (historical) sar -u -r -d 1 5
```

7. Service Diagnostics

- Use logs and systemd tools to diagnose service issues.
- Check service sttus, logs and configuration.
- Look for errors, restarts or dependency problems.

```
# Check service status
sudo systemctl status nginx
# View service logs
journalctl -u nginx -f
# Check failed services
systemctl --failed
# List recent restarts
journalctl -unginx --since "1 hour ago"
```

8. Investigating Production Incidents

- Identify the scope and symptoms.
- Check logs, metrics and service status.
- Look for errors, resource exhaustion, network issues or configuration changes.
- Correlate events using timestamps.

1. Identify the problem (What, When, Impact)
2. Check logs (journalctl, /var/log, dmesg)
3. Check metrics (CPU, memory, disk, network)
4. Find root cause (configuration, code, infrastructure)
5. Apply fix and monitor (verify and prevent recurrence)

9. Useful Commands and Log Locations

Purpose	Command / Location
View system logs	/var/log/messages or /var/log/syslog
View auth logs	/var/log/auth.log or /var/log/secure
View service logs	journalctl -u <service>
View kernel logs	dmesg or journalctl -k
View logs in real time	tail -f <logfile> or journalctl -f
Log rotation config	/etc/logrotate.conf /etc/logrotate.d/
System metrics	top, htop, free, df, iostat, ss, sar
Check failed services	systemctl --failed
Search logs	grep, journalctl --grep, rg
Filter by time	journalctl --since "YYYY-MM-DD HH:MM"
View log priority	journalctl -p err|warning|info

13. CPU SCHEDULING, MEMORY, AND RESOURCE MANAGEMENT

CPU utilization, load averages, process scheduling, virtual memory, page cache, swapping, out-of-memory (OOM) behavior, cgroups, and resource pressure analysis.

1. CPU Utilization and Load Averages

- CPU utilization shows how busy the CPU is.
- Load average is the average number of runnable processes (not CPU usage %).
- High load can be caused by CPU, I/O or memory pressure.
- Use top, htop, vmstat and uptime to analyze.

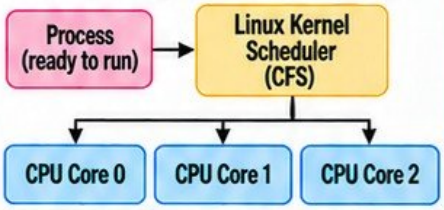
```
# Check load average
uptime

# Real-time CPU usage
top
htop

# Detailed CPU statistics
vmstat 1 5
mpstat -P ALL 1 5
```

2. Process Scheduling

- The kernel scheduler decides which process runs on the CPU.
- Processes have priorities (nice value).
- Different scheduling policies (CFS, FIFO, RR) are used.
- Use nice, renice and chrt to manage priorities.



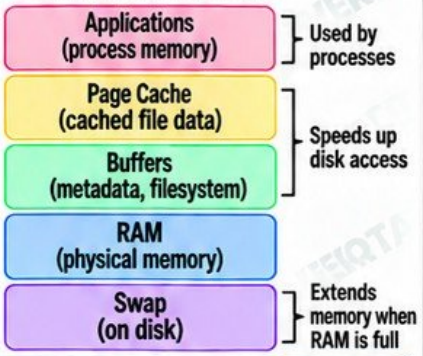

```
# Check process priority
ps -eo pid,comm,ni,pri --sort=-pri

# Change priority
nice -n 10 ./script.sh
renice -n -5 -p 1234

# Real-time scheduling (example)
chrt -f 50 ./app
```

3. Memory Management Basics

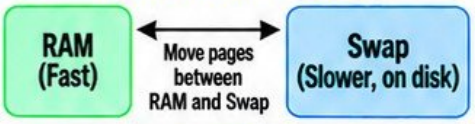
- Linux uses virtual memory.
- Includes RAM, page cache, buffers and swap.
- Page cache improves performance by caching disk data.
- Use free, top, htop and vmstat to analyze memory usage.




```
# Check memory usage
free -h
# Detailed memory information
cat /proc/meminfo
# Real-time view
vmstat 1 5
```

4. Swapping and Virtual Memory

- Swap is used when RAM is full.
- Helps prevent out-of-memory (OOM) situations.
- Excessive swapping causes poor performance.
- Use swapon, swapoff and vmstat to analyze.




```
# Check swap usage
free -h

# Enable swap (example)
sudo swapon /dev/sdb1

# Disable swap
sudo swapoff /dev/sdb1

# Monitor swapping activity
vmstat 1 5 | awk '{print $1,$3,$4}'
```

5. Out-of-Memory (OOM) Behavior

- When the system runs out of memory, the OOM killer terminates processes.
- Check kernel logs to see which process was killed.
- Prevent OOM by proper resource management and limits.

```
# Check recent OOM events
dmesg | grep -i 'oom'
# View kernel logs for OOM kills
journalctl -k | grep -i 'oom'

# Check system OOM killer setting
cat /proc/sys/vm/oom_kill_allocating_task

# Example log output
[ 12345.678] Out of memory: Killed process
1234 (nginx) total-vm:102400kB, anon-rss:...
[ 12345.678] oom-kill: constraint=CONSTRAINT_
MEM CGROUP,task=nginx, pid=1234, uid=0
```

6. cgroups and Resource Limits

- Control and limit CPU, memory and other resources.
- Used by containers and system services.
- Helps prevent resource exhaustion.
- Use systemd or cgroups directly.

```
# View cgroups (systemd)
systemd-cgls
# Show resource usage for a service
systemctl status nginx
systemd-cgtop
# Example: limit memory for a service
sudo systemctl set-property nginx \
  MemoryMax=500M
# View cgroup info
cat /sys/fs/cgroup/memory/nginx/
    memory.current
cat /sys/fs/cgroup/cpu/nginx/
    cpu.stat
```

7. Resource Pressure Analysis

- Identify CPU, memory, I/O bottlenecks.
- Use tools like top, htop, vmstat, iostat, sar and pressure stall information (PSI).
- Look for high load, high I/O wait or memory pressure.

```
# CPU, memory and I/O overview
vmstat 1 5
iostat -x 1 5
sar -u 1 5   # CPU usage
sar -r 1 5   # memory usage
sar -d 1 5   # disk I/O
```

```
# Check pressure stall information (PSI)
cat /proc/pressure/cpu
cat /proc/pressure/memory
cat /proc/pressure/io
# Example output
you can see some avg10=0.42 avg60=0.35 avg300=0.20
full avg10=0.10 avg60=0.05 avg300=0.01
```

8. Key Takeaways

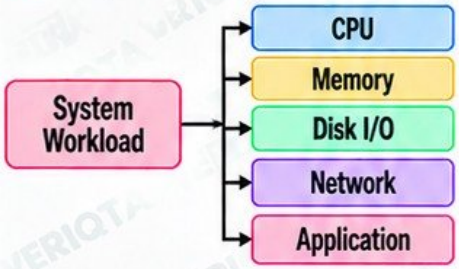
- Monitor CPU, memory, swap and I/O.
- Understand how the kernel schedules processes.
- Use the right tools to identify bottlenecks.
- Prevent and handle OOM situations.
- Use cgroups and limits to control resources.
- Regularly analyze system performance.

14. LINUX PERFORMANCE ANALYSIS AND BOTTLENECK INVESTIGATION

Performance troubleshooting using vmstat, iostat, pidstat, sar, perf, and pressure stall information (PSI). Distinguishing CPU, memory, disk, and network bottlenecks.

1. Performance Analysis Overview

- Identify performance bottlenecks.
- Monitor system resources in real time and over time.
- Use the right tools for CPU, memory, disk and network analysis.
- Correlate metrics to find the real cause.
- Compare normal vs abnormal behaviour.



2. CPU Performance Analysis

- Check CPU utilization, run queue and context switching.
- Identify CPU-bound processes.
- Use vmstat, pidstat, sar and perf.

```
# View CPU and system stats
vmstat 1 5
procs -----------memory---------- ---swap--
r  b   swpd   free    buff  cache   si   so  us
1  0      0 205324  12344 456788    0    0  12
0  0      0 204980  12360 456912    0    0   8

# Per-process CPU usage
pidstat -u 1 3
PID    %usr %system  %guest   %CPU   Command
1234  12.34   0.56    0.00   12.90   java
5678   1.23   0.10    0.00    1.33   nginx
```

3. Memory Performance Analysis

- Monitor memory usage, swapping and page cache.
- Identify memory pressure and leaks.
- Use vmstat, free, sar and PSI.

```
# Memory usage overview
free -h
              total        used        free      shared  buff/cache
Mem:          7.7G        6.1G        512M        328M        1.1G
Swap:         2.0G        256M        1.7G

# Memory stats over time
vmstat 1 5
 si   so    bi    bo   us   sy   id   wa   st
  0    0    12    48   10    5   84    1    0
  0    0     8    32   12    6   80    2    0

# Check memory pressure (PSI)
cat /proc/pressure/memory
some  avg10=0.12 avg60=0.08 avg300=0.05
full  avg10=0.00 avg60=0.00 avg300=0.00
```

4. Disk I/O Performance Analysis

- Check disk read/write performance.
- Identify high I/O, high latency or queue length issues.
- Use iostat, pidstat and sar.

```
# Disk I/O statistics
iostat -xz 1 3
Device   wrqm/s   wrqm/s    r/s     w/s    rMB/s
sda        0.00     1.20  12.50   45.30     1.8
sdb        0.10     0.30   0.50  120.00    15.6

# Per-process I/O usage
pidstat -d 1 3
PID      kB_rd/s      kB_wr/s      Command
2345      120.00       560.00      mysqld
6789        0.00       320.00      java
```

5. Network Performance Analysis

- Monitor network throughput, connections and errors.
- Identify network bottlenecks.
- Use sar, pidstat and ss.

```
# Network stats with sar
sar -n DEV 1 3
IFACE   rxkB/s   txkB/s   rxpck/s   txpck/s
eth0    1024.5    856.3    1200.1    1100.4
eth1      12.3      8.6     100.2      90.1

# Per-process network usage
pidstat -n 1 3
PID      kB_rd/s      kB_wr/s      Command
4321      450.00       320.00      nginx
8765      120.00       980.00      java

# Check listening sockets
ss -tulnp | head -5
State      Local Address          Process
LISTEN     0.0.0.0:22             sshd
LISTEN     0.0.0.0:80             nginx
LISTEN     127.0.0.1:3306         mysqld
```

6. Using perf for Deep Analysis

- Profiling CPU hotspots and function calls.
- Useful for advanced performance investigation.
- Identify what is consuming CPU.

```
# Record performance data
sudo perf record -g -p <pid> -- sleep 30

# View top functions
sudo perf report --stdio | head -10
Samples   %      Command   Function
3456     35.2    java      [.] do_futex
2100     21.4    java      [.] tcp_recvmsg
1200     12.3    java      [.] schedule
980      10.0    java      [.] tcp_sendmsg
```

7. Pressure Stall Information (PSI)

- Shows resource pressure for CPU, memory and I/O.
- Helps detect contention before it impacts performance.
- Useful for monitoring and alerting.

```
# Check CPU, memory and I/O pressure
cat /proc/pressure/cpu
some  avg10=0.05 avg60=0.03 avg300=0.01
full  avg10=0.00 avg60=0.00 avg300=0.00
cat /proc/pressure/memory
some  avg10=0.12 avg60=0.08 avg300=0.05
full  avg10=0.00 avg60=0.00 avg300=0.00
cat /proc/pressure/io
some  avg10=0.08 avg60=0.06 avg300=0.04
full  avg10=0.00 avg60=0.00 avg300=0.00
```

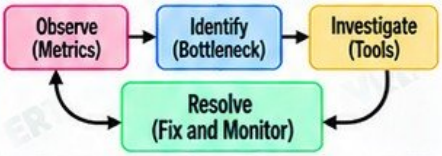
8. Distinguishing Bottlenecks

- Use metrics and symptoms to identify the bottleneck.
- Check multiple resources and correlate results.

Bottleneck	Symptoms	Key Tools
CPU	High CPU usage, high run queue	top, htop, vmstat, pidstat, perf, sar
Memory	High memory usage, swapping, OOM	free, vmstat, sar, PSI
Disk I/O	High I/O wait, high latency, high queue	iostat, pidstat, sar
Network	High network usage, packet drops, high latency	sar, pidstat, ss, netstat

9. Key Takeaways

- Use the right tool for the right resource.
- Look at trends, not just snapshots.
- Correlate metrics to find the root cause.
- Distinguish between CPU, memory, disk and network bottlenecks.
- Use PSI to detect early signs of pressure.
- Combine tools for deeper analysis.
- Understand normal vs abnormal patterns.
- Document findings and apply fixes systematically.

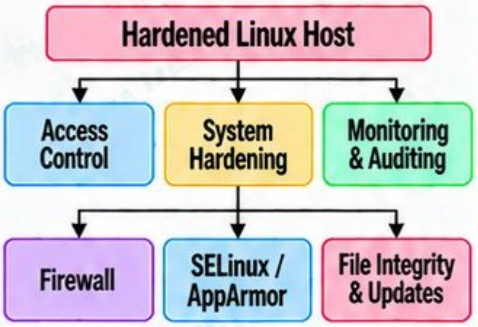


15. LINUX SECURITY HARDENING AND HOST PROTECTION

SELinux, AppArmor, firewall management, auditd, kernel security controls, sudo policies, SSH hardening, file integrity, and reducing production attack surfaces.

1. Security Hardening Overview

- Reduce the system attack surface.
- Apply least privilege principle.
- Enable and configure security controls.
- Monitor and audit system activities.
- Regularly update and patch the system.



2. SELinux (RHEL/CentOS)

- Mandatory Access Control (MAC).
- Restricts what processes can do.
- Helps contain damage if compromised.
- Use enforcing mode in production.

```
# Check SELinux status
sestatus
# View current mode
getenforce
# Set enforcing mode (temporarily)
sudo setenforce 1
# Make it permanent
sudo sed -i 's/^SELINUX=.*/SELINUX=enforcing/' /etc/selinux/config
# View SELinux denials
sudo ausearch -m avc -ts recent
# Analyze denial (install setroubleshoot)
sudo sealert -a /var/log/audit/audit.log
```

3. AppArmor (Ubuntu/Debian)

- Mandatory Access Control (MAC).
- Profiles restrict application behaviour.
- Helps limit the impact of attacks.
- Use official profiles and custom profiles.

```
# Check AppArmor status
sudo aa-status
# View profiles
sudo aa-list
# Check if a process is confined
sudo aa-status | grep nginx
# Put a profile in enforce mode
sudo aa-enforce /etc/apparmor.d/usr.sbin.nginx
# Put a profile in complain mode
sudo aa-complain /etc/apparmor.d/usr.sbin.nginx
# Reload AppArmor profiles
sudo systemctl reload apparmor
```

4. Firewall Management

- Control inbound and outbound traffic.
- Allow only necessary services.
- Use firewalld (RHEL/CentOS) or ufw (Ubuntu).
- Regularly review and update rules.

firewalld (RHEL/CentOS)

```
sudo firewall-cmd --state
sudo firewall-cmd --list-all
sudo firewall-cmd --add-service=ssh --permanent
sudo firewall-cmd --reload
```

ufw (Ubuntu/Debian)

```
sudo ufw status
sudo ufw allow ssh
sudo ufw allow 80
sudo ufw allow 443
sudo ufw deny 22
sudo ufw enable
```

5. auditd and System Auditing

- Track system events and user activities.
- Monitor file access, privilege changes, and security events.
- Useful for incident investigation.

```
# Check auditd status
sudo systemctl status auditd
# View recent audit logs
sudo ausearch -ts recent
# Search for specific events
sudo ausearch -m USER_CMD
# View logs in readable format
sudo aureport -x --summary
# Monitor configuration changes
sudo ausearch -m CONFIG_CHANGE
# Audit rules configuration file
cat /etc/audit/rules.d/audit.rules
```

6. Kernel Security Controls

- Use sysctl to configure kernel parameters.
- Disable unnecessary features.
- Enable protections like ASLR, kernel pointer restriction, and core dump control.

```
# View current kernel parameters
sysctl -a | grep -E 'kptr|dmesg|core|randomize'
# Enable ASLR (Address Space Layout Randomization)
sudo sysctl -w kernel.randomize_va_space=2
# Restrict kernel pointer exposure
sudo sysctl -w kernel.kptr_restrict=2
# Disable core dumps (optional)
sudo sysctl -w kernel.core_pattern=|/bin/false
# Make changes permanent
echo 'kernel.randomize_va_space = 2' | sudo tee -a /etc/sysctl.conf
```

7. Sudo Policies and Privilege Management

- Use sudo instead of direct root login.
- Apply least privilege principle.
- Restrict sudo access to specific users/commands.
- Use sudoers file for fine-grained control.

```
# Edit sudoers file safely
sudo visudo
# Example: allow user to run specific commands
devops ALL=(root) /usr/bin/systemctl
# Require password for sudo (default)
Defaults    requiretty
# Add command logging
Defaults    logfile="/var/log/sudo.log"
```

8. SSH Hardening

- Disable root login.
- Use key-based authentication.
- Change default port (optional).
- Restrict access with AllowUsers.
- Use strong ciphers and modern settings.

```
# Edit SSH configuration
sudo vim /etc/ssh/sshd_config
# Key settings
PermitRootLogin no
PasswordAuthentication no
AllowUsers devops
Port 2222
PubkeyAuthentication yes
# Apply changes
sudo systemctl restart sshd
```

9. File Integrity and Attack Surface Reduction

- Monitor critical system files for changes.
- Use tools like AIDE or Tripwire.
- Remove unnecessary packages and services.
- Disable unused network services and ports.

```
# Install and initialize AIDE (RHEL/Ubuntu)
sudo apt install aide    # or: sudo dnf install aide
sudo aide --init
sudo mv /var/lib/aide/aide.db.new /var/lib/aide/aide.db
# Run file integrity check
sudo aide --check
# List running services (disable unused ones)
systemctl list-units --type=service --state=running
# List open ports
ss -tulnp
```

10. Key Takeaways

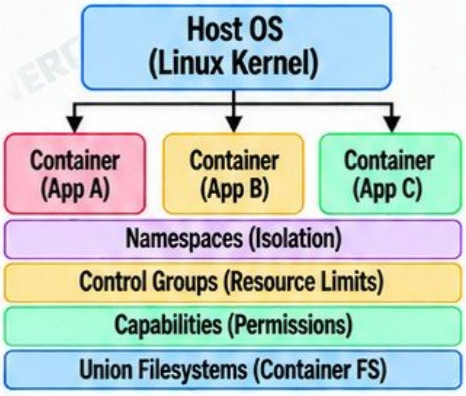
- Enable and properly configure SELinux or AppArmor.
- Use a firewall to allow only required traffic.
- Monitor and audit system activities with auditd.
- Apply secure kernel parameters.
- Use sudo with least privilege.
- Harden SSH and use key-based authentication.
- Implement file integrity monitoring.
- Remove unnecessary software and services.
- Continuously reduce the attack surface.

16. LINUX CONTAINERS, NAMESPACES, AND CONTROL GROUPS

Process isolation, PID namespaces, network namespaces, mount namespaces, cgroups v2, container filesystems, capabilities, and how Linux enables Docker and Kubernetes.

1. Containers and Linux Isolation

- Containers use Linux kernel features for isolation.
- Share the host kernel but run in isolated environments.
- Provide lightweight, fast and scalable application deployment.
- Built using namespaces, cgroups, capabilities and filesystem isolation.



2. PID Namespaces

- Isolate process IDs.
- Each container has its own process tree.
- PID 1 inside a container is isolated from the host.
- Prevents containers from seeing host processes.

```
# Start a process in a new PID namespace
sudo unshare -pf --mount-proc bash

# Inside the namespace
ps -ef

# PID 1 is the shell in this namespace
echo "My PID: $$"
ps -ef

# From the host, the process is different
ps -ef | grep unshare
```

3. Network Namespaces

- Isolate network devices, IP addresses, routing tables and firewall rules.
- Each container has its own network stack.
- Containers can have virtual network interfaces (veth).
- Used by Docker and Kubernetes for network isolation.

```
# Create a new network namespace
sudo ip netns add testns
# Add a veth pair
sudo ip link add veth0 type veth \
   peer name veth1
# Move one end to the namespace
sudo ip link set veth1 netns testns
# Assign IP addresses
sudo ip addr add 10.0.0.1/24 dev veth0
sudo ip netns exec testns ip addr add \
  10.0.0.2/24 dev veth1
# Bring interfaces up
sudo ip link set veth0 up
sudo ip netns exec testns ip link set lo up
```

4. Mount Namespaces

- Isolate the filesystem view.
- Each container has its own mount points.
- Allows containers to have their own root filesystem (e.g. overlayfs).
- Prevents containers from seeing or modifying host mounts.

```
# Start a new mount namespace
sudo unshare -m bash

# Create and mount a new filesystem
sudo mount -t tmpfs tmpfs /mnt

# Verify mounts (only visible in namespace)
mount | grep /mnt

# Create a private directory structure
mkdir -p /mnt/containerroot
ls /mnt
```

5. Control Groups (cgroups v2)

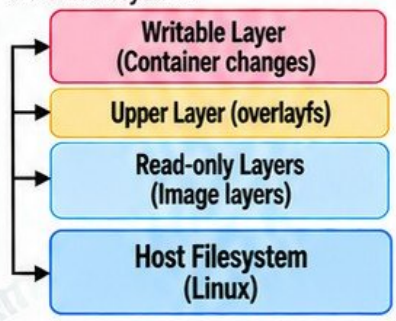
- Limit and monitor resource usage (CPU, memory, I/O, etc).
- Prevent a container from consuming all system resources.
- Used by Docker, Kubernetes and systemd.
- cgroups v2 provides a unified hierarchy and better resource controls.

```
# Check cgroups v2 (unified hierarchy)
mount | grep cgroup
ls /sys/fs/cgroup

# Create a cgroup with memory limit
sudo mkdir /sys/fs/cgroup/mycontainer
echo "500M" | sudo tee \
  /sys/fs/cgroup/mycontainer/memory.max
# Run a process in the cgroup
sudo systemd-run --scope -p \
  MemoryMax=500M bash
```

6. Container Filesystems

- Containers use layered filesystems (e.g. overlayfs).
- Provide a writable layer on top of read-only layers.
- Efficient storage and fast image distribution.
- Changes in a container do not affect the host filesystem.



7. Linux Capabilities

- Break root privileges into smaller permissions.
- Containers run with limited capabilities by default.
- Helps reduce the attack surface.
- Only the required capabilities are granted.

```
# View process capabilities
capsh --print
# Run a container with specific capability
docker run --cap-add NET_ADMIN \
  --cap-drop ALL nginx
# List all capabilities
capsh --decode=0x00000000a80425fb
```

8. How Linux Enables Docker and Kubernetes

- Docker uses namespaces for isolation.
- Uses cgroups for resource limits.
- Uses capabilities for security.
- Kubernetes builds on these same features and adds orchestration, networking and scheduling.



Docker
(uses Linux namespaces, cgroups, capabilities)

Kubernetes
(uses the same Linux features at scale)

9. Key Takeaways

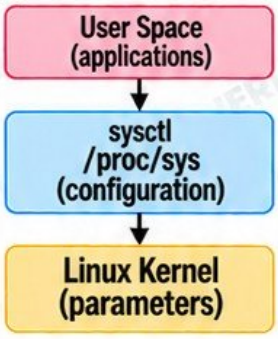
- Containers are not magic; they use Linux kernel features.
- Namespaces provide isolation (PID, network, mount, etc).
- cgroups control resource usage (CPU, memory, I/O).
- Capabilities limit privileges.
- Container filesystems use layered storage.
- These features enable Docker and Kubernetes.
- Understanding these concepts helps in troubleshooting, security hardening and production operations.

17. LINUX KERNEL PARAMETERS AND PRODUCTION TUNING

sysctl, /proc/sys, file descriptor limits, process limits, TCP backlog settings, memory tuning, persistent configuration, and validating changes safely.

1. Kernel Parameters Overview

- Control Linux behaviour at runtime.
- Tune for better performance, scalability and stability.
- Configured using sysctl, /proc/sys or configuration files.
- Commonly tuned in production for networking, memory and file limits.



2. Using sysctl and /proc/sys

- sysctl is a user-friendly command to view and set kernel parameters.
- /proc/sys exposes the same parameters as files.
- Changes with sysctl are temporary (until reboot).

```
# View a parameter
sysctl net.ipv4.ip_forward
# Set a parameter (temporary)
sudo sysctl -w net.ipv4.ip_forward=1
# Same using /proc/sys
cat /proc/sys/net/ipv4/ip_forward
echo 1 | sudo tee \\
  /proc/sys/net/ipv4/ip_forward
# View all parameters
sysctl -a
# Filter specific parameters
sysctl -a | grep net.ipv4
```

3. File Descriptor Limits

- Control the number of open files per process and system-wide.
- Important for high-concurrency applications (web servers, databases).
- Configured using ulimit and /etc/security/limits.conf.

```
# Check current limits
ulimit -n        # per process
cat /proc/sys/fs/file-max  # system-wide
# Set temporary limit
ulimit -n 65536
# Configure permanent limits
# /etc/security/limits.conf
* soft    nofile   65536
* hard    nofile   65536
# Apply changes (new session required)
source /etc/security/limits.conf
```

4. Process Limits

- Control the number of processes, memory usage and other resources.
- Useful to prevent a single application from exhausting system resources.
- Configured using ulimit and /etc/security/limits.conf.

```
# View current process limits
ulimit -a
# Example output (common)
max user processes (-u)  4096
open files (-n)          1024
max memory size (-m)     unlimited
# Set a process limit (temporary)
ulimit -u 8192
# Permanent configuration
# /etc/security/limits.conf
*  soft   nproc   8192
*  hard   nproc   8192
```

5. TCP Backlog and Network Tuning

- Tune TCP settings for high traffic and high concurrency.
- Adjust backlog queues, connection handling and timeouts.
- Helps improve performance and prevent connection drops.

```
# View TCP parameters
sysctl net.core.somaxconn
sysctl net.ipv4.tcp_max_syn_backlog
# Set TCP backlog (temporary)
sudo sysctl -w net.core.somaxconn=65535
sudo sysctl-w net.ipv4.tcp_max_syn_backlog=65535
# Common network tuning parameters
sudo sysctl -w net.ipv4.tcp_fin_timeout=15
sudo sysctl -w net.ipv4.tcp_keepalive_time=300
sudo sysctl -w net.ipv4.tcp_tw_reuse=1
sudo sysctl -w net.ipv4.ip_local_port_range=
              "1024 65535"
```

6. Memory Tuning

- Optimize kernel memory behaviour for your workload.
- Tune swappiness, cache behaviour and memory overcommit settings.
- Helps improve stability and avoid out-of-memory conditions.

```
# Check memory parameters
sysctl vm.swappiness
sysctl vm.overcommit_memory
sysctl vm.dirty_ratio

# Set memory tuning parameters
sudo sysctl -w vm.swappiness=10
sudo sysctl -w vm.overcommit_memory=1
sudo sysctl -w vm.dirty_ratio=10
sudo sysctl -w vm.dirty_background_ratio=5
```

7. Persistent Configuration

- Use /etc/sysctl.conf or files in /etc/sysctl.d/ for permanent settings.
- Changes in these files are applied at boot time.
- Use descriptive filenames and document the reason for each change.

```
# Edit sysctl configuration file
sudo nano /etc/sysctl.conf
# Example settings
net.ipv4.ip_forward = 1
net.core.somaxconn = 65535
vm.swappiness = 10
# Apply changes immediately
sudo sysctl -p
```

8. Validating Changes Safely

- Check current values before and after making changes.
- Test changes in a controlled manner.
- Monitor system behaviour for any negative impact.
- Use rollback if needed.

```
# Check current value
sysctl parameter.name
# Apply change and verify
sudo sysctl -w parameter.name=value
sysctl parameter.name
# Monitor system after changes
top, htop, vmstat, ss, netstat
# Rollback to original value if needed
sudo sysctl -w parameter.name=original_value
```

9. Key Takeaways

- Kernel parameters affect system performance, scalability and stability.
- Use sysctl and /proc/sys to view and set parameters.
- Tune file descriptor and process limits for high-concurrency workloads.
- Adjust TCP and memory settings based on your environment.
- Make changes persistent using /etc/sysctl.conf or /etc/sysctl.d/.
- Always validate changes safely and monitor system behaviour.
- Document your tuning decisions.

18. LINUX PRODUCTION TROUBLESHOOTING AND INCIDENT RESPONSE

Diagnosing unresponsive servers, failed services, disk-full conditions, high load, memory exhaustion, DNS failures, permission errors, and network connectivity incidents using evidence-driven investigation.

1. Troubleshooting Process Overview

- Stay calm and understand the impact.
- Gather evidence before making changes.
- Identify symptoms, isolate the cause.
- Use the right tools and logs.
- Apply fix, validate, and monitor.
- Document what happened.



2. Unresponsive Server Troubleshooting

- Check if the server is reachable.
- Check system load and resource usage.
- Identify hung processes or kernel issues.
- Review recent logs and system events.

```
# Check server responsiveness
ping -c 4 <server-ip>
# Check system load
uptime
top
# Check running processes
ps -eo pid,ppid,cmd,%cpu,%mem --sort=-%cpu
# Check for kernel or hardware issues
dmesg | tail -n 50
journalctl -xe --since "1 hour ago"
# Identify blocked processes
ps -eo state,pid,cmd | grep '^D'
```

3. Failed Services Investigation

- Check service status and logs.
- Identify dependency failures.
- Look for configuration or permission issues.
- Verify resource availability.

```
# Check service status
systemctl status <service>
# View recent service logs
journalctl -u <service> -n 50
# Follow logs in real time
journalctl -u <service> -f
# Check service dependencies
systemctl list-dependencies <service>
# Common checks
ss -tulnp | grep <port>
ps -ef | grep <service>
ls -l /etc/<service>/
```

4. Disk-Full Condition Troubleshooting

- Identify which partition is full.
- Find large files and directories.
- Check for log files, core dumps or temporary files.
- Clean up safely without deleting important data.

```
# Check disk usage
df -h
# Find large directories
du -sh /* 2>/dev/null | sort -h
# Find large files (top 10)
find / -type f -size +100M 2>/dev/null \
 | sort -h | tail -n 10
# Check inode usage
df -i
# Check and clean old logs
journalctl --disk-usage
journalctl --vacuum-time=7d
```

5. High Load Troubleshooting

- Identify what is consuming CPU, memory, I/O or causing process contention.
- Check running processes and services.
- Look for traffic spikes or background jobs.

```
# Check system load
uptime
top
# Detailed process view
htop
# CPU usage by process
ps -eo pid,cmd,%cpu --sort=-%cpu | head
# Check I/O wait and disk usage
iostat -xz 1 5
# Check memory usage
free -h
# Check network usage
sar -n DEV 1 5
```

6. Memory Exhaustion Troubleshooting

- Check memory usage and swap.
- Find processes using high memory.
- Look for memory leaks or runaway processes.
- Analyse application logs and behaviour.

```
# Check memory usage
free -h
# Top memory consuming processes
ps -eo pid,cmd,%mem --sort=-%mem | head
# Detailed view
htop
# Check for out-of-memory (OOM) events
dmesg | grep -i oom
journalctl -k | grep -i oom
# Check swap usage
swapon --show
# Check memory pressure (PSI)
cat /proc/pressure/memory
```

7. DNS Failure Troubleshooting

- Check name resolution.
- Verify DNS server settings.
- Test with different DNS servers.
- Look for network or firewall issues.

```
# Test name resolution
nslookup <domain>
dig <domain>
# Check DNS configuration
cat /etc/resolv-conf
systemd-resolve --status
# Test with public DNS
dig @8.8.8.8 <domain>
# Check connectivity to DNS server
ping <dns-server-ip>
```

8. Permission Errors Troubleshooting

- Check file ownership and permissions.
- Verify user and group access.
- Look for SELinux/AppArmor denials.
- Identify incorrect sudo configuration.

```
# Check file permissions
ls -l <file-or-directory>
# Check ownership
ls -la <file-or-directory>
# Check current user and groups
id
groups
# Check SELinux denials (RHEL/CentOS)
ausearch -m avc -ts recent
# Check AppArmor denials (Ubuntu)
dmesg | grep -i apparmor
```

9. Network Connectivity Issues

- Check connectivity to hosts and ports.
- Verify routing and firewall rules.
- Test DNS and name resolution.
- Check listening services and interfaces.

```
# Test host connectivity
ping -c 4 <host>
# Test specific port
nc -zv <host> <port>
# Check listening ports
ss -tulnp
# Check network interfaces
ip addr show
# Check routing
ip route
# Check firewall rules (RHEL/CentOS)
sudo firewall-cmd --list-all
# Check firewall rules (Ubuntu)
sudo ufw status
```

10. Key Takeaways

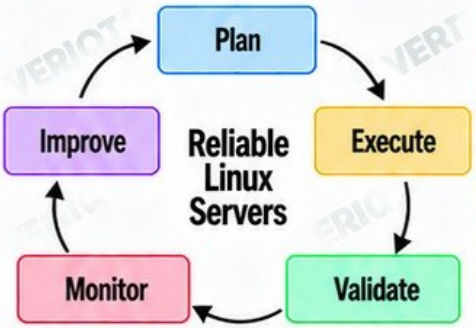
- Use a structured and evidence-driven approach.
- Check logs, metrics and system state before making changes.
- Identify the real root cause, not just the symptoms.
- Know the right tools for each type of issue.
- Validate the fix and monitor afterwards.
- Document the incident and lessons learned.
- Practice these troubleshooting scenarios regularly.

19. LINUX SERVER MAINTENANCE, PATCHING, AND RELIABILITY ENGINEERING

Kernel updates, service maintenance, reboot coordination, redundant capacity, configuration management, rollback planning, backup verification, and minimizing production disruption.

1. Server Maintenance Overview

- Keep systems secure, stable and reliable.
- Plan maintenance to minimize downtime.
- Coordinate changes with proper communication.
- Validate systems before and after maintenance.
- Automate routine maintenance tasks.



2. Kernel Updates and System Patching

- Keep the kernel and packages up to date.
- Apply security patches regularly.
- Check for required reboots after updates.
- Test updates in a staging environment when possible.
- Monitor for issues after applying patches.

```
# Check available updates (Ubuntu/Debian)
sudo apt update && sudo apt list --upgradable
# Check available updates (RHEL/CentOS)
sudo dnf check-update
# Show current kernel version
uname -r
# Install security updates (Ubuntu/Debian)
sudo apt install --only-upgrade
# Install security updates (RHEL/CentOS)
sudo dnf update --security
# Check if reboot is required (Ubuntu/Debian)
sudo needrestart
# Check if reboot is required (RHEL/CentOS)
sudo rpm -q --last kernel
```

3. Service Maintenance

- Check service status before and after maintenance.
- Restart services safely.
- Use systemd best practices.
- Perform zero-downtime reloads when supported.
- Verify service dependencies.

```
# Check service status
systemctl status nginx
# Reload service without downtime
sudo systemctl reload nginx
# Restart service (when required)
sudo systemctl restart nginx
# Enable service at boot
sudo systemctl enable nginx
# View service logs
journalctl -u nginx -f
# Check failed services
systemctl --falled
```

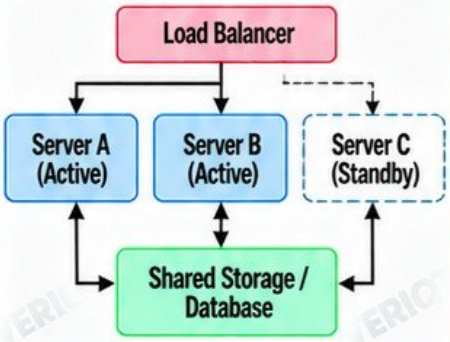
4. Reboot Coordination

- Determine if a reboot is required.
- Schedule reboots during maintenance windows.
- Notify stakeholders before rebooting.
- Verify all services are running after reboot.
- Use high availability to minimize impact.

```
# Check uptime and last reboot
uptime
who -b
# Check if reboot is required
sudo needrestart
# Reboot the server (planned)
sudo reboot
# Cancel a scheduled reboot
sudo shutdown-c
# Schedule a reboot at a specific time
sudo shutdown -r +30"Planned reboot"
```

5. Redundant Capacity and High Availability

- Maintain redundant servers and capacity.
- Use load balancers and clustering.
- Perform rolling updates.
- Ensure automatic failover.
- Test failover procedures regularly.



6. Configuration Management

- Keep server configurations consistent.
- Use configuration management tools (Ansible, etc).
- Track and version configuration changes.
- Apply changes in a controlled manner.
- Detect and revert unwanted changes.

```
# Example: check SSH configuration
cat /etc/ssh/sshd_config
# Test SSH configuration before reload
sudo sshd -t
# Use configuration management (Ansible)
ansible all -m ping
ansible-playbook site.yml
# Track configuration changes with Git
git status
git commit -m "Update nginx config"
# Compare current and backup config
diff /etc/nginx/nginx.conf \
     /etc/nginx/nginx.conf.bak
```

7. Rollback Planning

- Have a rollback plan before making changes.
- Keep previous kernel and package versions.
- Backup configuration files.
- Test rollback procedures.
- Document the rollback process.

```
# List installed kernel versions
rpm -q kernel  # RHEL/CentOS
dpkg --list | grep linux-image  # Ubuntu

# Boot into previous kernel (GRUB menu)
# Example: set default kernel (GRUB)
sudo grub2-set-default "1"  # RHEL/CentOS
sudo update-grub            # Ubuntu
```

8. Backup Verification

- Verify backups are running successfully.
- Test restore procedures regularly.
- Check backup integrity.
- Store backups in separate locations.
- Monitor backup storage capacity.

```
# Check backup service status
systemctl status backup.service
# Verify backup files
ls -lh /backup
# Test archive integrity
tar -tzf /backup/server-$(date +%F).tar.gz
# Test restore (example)
tar -xzf /backup/server-2026-10-01.tar.gz \
   -C /tmp/restore-test
# Check backup storage usage
df -h /backup
```

9. Key Takeaways

- Keep systems updated and secure.
- Plan and communicate maintenance activities.
- Use redundant capacity and high availability.
- Manage configurations and track changes.
- Have a rollback plan.
- Verify backups regularly.
- Minimize production disruption.
- Monitor and validate after changes.



20. SENIOR LINUX ENGINEER PRACTICAL ASSESSMENT

Advanced production scenarios covering failed services, storage exhaustion, performance degradation, security misconfiguration, networking failures, and incident recovery. Includes diagnostic commands, root-cause reasoning, remediation decisions, and verification.

1. Failed Services Scenario

- A critical service is not starting or keeps crashing.
- Investigate logs, dependencies and resource issues.
- Identify the root cause and fix it.
- Verify the service is stable.

Common Diagnostic Commands

```
systemctl status <service>
journalctl -u <service> -n 100
systemctl list-dependencies <service>
ss -tulnp | grep <port>
dmesg | tail -n 50
free -h && df -h
ps aux | grep <process>
```

2. Storage Exhaustion Scenario

- The server is running out of disk space.
- Find what is consuming the space.
- Clean up safely without causing production issues.
- Implement a long-term solution.

Find and Investigate Disk Usage

```
df -h
du -sh /* 2>/dev/null | sort -h
du -sh /var/ * 2>/dev/null | sort -h
find / -type f -size +100M 2>/dev/null
lsof +L1
journalctl --disk-usage
docker system df   # if using Docker
```

3. Performance Degradation Scenario

- The server is slow or unresponsive.
- Identify whether the issue is CPU, memory, disk or network related.
- Find the cause using system metrics.
- Take appropriate remediation steps.

Check System Performance

```
top
htop
vmstat 1 5
iostat -xz 1 5
pidstat -u -r -d 1 5
sar -n DEV 1 5
perf top
cat /proc/pressure/{cpu,memory,io}
```

4. Security Misconfiguration Scenario

- A security issue or misconfiguration is detected.
- Investigate and assess the risk.
- Apply the correct fix and hardening.
- Verify that the system is secure.

Check Security Configuration

```
sudo -l
getenforce  # SELinux status
aa-status  # AppArmor status
ufw status  # Firewall (Ubuntu)
firewall-cmd --list-all  # RHEL/CentOS
sshd -T
grep -E 'PermitRootLogin|PasswordAut'
 /etc/ssh/sshd_config
ausearch -m avc -ts recent # Audit logs
```

5. Networking Failure Scenario

- The server cannot reach other systems or external services.
- Diagnose DNS, routing, firewall and connectivity issues.
- Identify the root cause and fix it.
- Verify end-to-end connectivity.

Network Troubleshooting Commands

```
ip addr show
ip route
ping -c 4 <host>
dig <domain>
nslookup <domain>
ss -tulnp
traceroute <host>
curl -v http://<host>
tcpdump -i <interface>
iptables -L -n | grep <port>
```

6. Memory Exhaustion Scenario

- The server is out of memory or applications are being killed.
- Identify memory usage and OOM events.
- Find the process or cause.
- Apply short-term and long-term fixes.

Investigate Memory Issues

```
free -h
top
ps aux --sort=-%mem | head
dmesg | grep -i 'oom'
journalctl -k | grep -i oom
vmstat 1 5
cat /proc/meminfo
cat /proc/pressure/memory
sar -r 1 5
```

7. Incident Recovery Scenario

- A critical incident has impacted production.
- Follow a structured investigation process.
- Restore service and verify stability.
- Document the incident and lessons learned.

Incident Response Steps

1. Assess the impact and scope.
2. Gather evidence (logs, metrics, configs).
3. Identify root cause.
4. Implement a fix or mitigation.
5. Verify service recovery.
6. Monitor for recurrence.
7. Document findings and actions.

8. Remediation and Verification

- Apply the correct fix based on the root cause.
- Verify the service or system is working as expected.
- Monitor to ensure the issue does not recur.
- Implement preventive measures.

Common Verification Commands

```
systemctl status <service>
curl -I http://<service-endpoint>
ss -tulnp | grep <port>
df -h
free -h
uptime
tail -n 50 /var/log/<service>.log
echo "Test successful"
```

9. Key Takeaways

- Use a systematic, evidence-driven approach.
- Always identify the root cause, not just the symptom.
- Validate the fix and monitor afterwards.
- Consider security, performance and reliability in all decisions.
- Document incidents and improvements.
- Build automation and monitoring to prevent recurrence.

